Version	0.16
Date	2026-04-13
Firmware	1.0.44

D21 eStation User Manual

D21

This document applies only to the integration of ETAG ESL Gen 3.0 and DSL Gen 1.0 systems

Technical Support： huanghaipengonline@hotmail.com

Limited Scope of Use

Revision History

Version	Date	Description	Author	Review
0.1	2024-08-07	Document Initialization	Haipeng	Haipeng
0.2	2024-08-13	Add description of Topic QoS levels on the device side	Haipeng	Haipeng
0.3	2024-08-14	Fixed an error in the data structure definition of TaskResult	Haipeng	Haipeng
0.4	2025-01-05	Add secure communication description and modified Pattern definition	Haipeng	Haipeng
0.5	2025-01-07	Add description of topic taskESL2	Haipeng	Haipeng
0.6	2025-01-08	Update description of topic taskESL2	Haipeng	Haipeng
0.7	2025-02-17	Add topic alias Add topic taskDSL and OTA	Haipeng	Haipeng
0.8	2025-03-01	Rename some topics, remove some invalid pattern	Haipeng	Haipeng
0.9	2025-03-25	Fixed an error in the security communication	Haipeng	Haipeng
0.10	2025-04-10	Fixed an error in the data structure definition of topic taskDSL Add RGB mixed color table	Haipeng	Haipeng
0.11	2025-04-30	Add description of LED turn on in topic taskDSL Add description of OTA MD5 value	Haipeng	Haipeng
0.12	2025-06-23	Corrected data type errors in data structure definitions	Haipeng	Haipeng
0.13	2025-07-26	Add topic taskDSL2	Haipeng	Haipeng
0.14	2025-08-12	Added description of ESL graphic dimensions Corrected minor errors	Haipeng	Haipeng
0.15	2025-08-19	Corrected minor errors Remove the Pattern.UpdateImage in DSLEntity	Haipeng	Haipeng
0.16	2026-04-13	Add 6 colors, 7 colors ESL support Add support for mainstream image formats	Haipeng	Haipeng

Content

1 Description 4
1.1 Background 4
1.2 System Architecture of eStation 6
1.3 Images 6
2 Use eStation 8
2.1 Connect to eStation 9

Limited Scope of Use

2.2 Receiving eStation Information 10
2.3 Receiving eStation Messages 10
2.4 Receiving eStation Results 11
2.5 Receiving eStation Heartbeat 12
2.6 Publishing Configuration Information 13
2.7 Publishing ESL Task（Base64 Version） 13
2.8 Publishing ESL Task（Bytes Version） 14
2.9 Publishing DSL Task 15
2.10 Publishing DSL Task （with Pattern and Key） 15
2.11 Publishing OTA Task（Firmware） 16
2.12 Publishing OTA Task（ESL） 17
2.13 Secure Communication 17
3 References 18
3.1 ESL Gen 3.0 Model List 18
3.2 Pattern 20
3.3 PageIndex 20
3.4 RGB Color Definitions 20
3.5 Data Definitions（C#） 20

D21

Limited Scope of Use

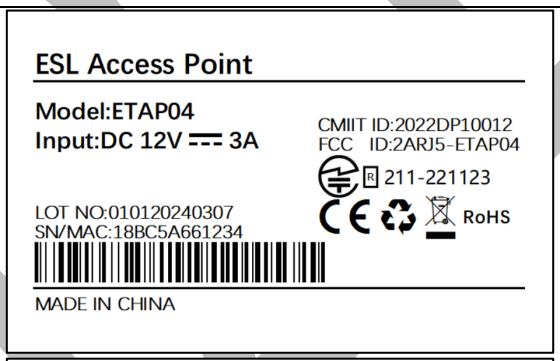
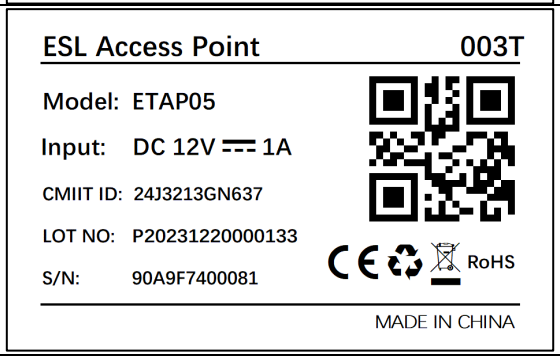
1 Description

1.1 Background

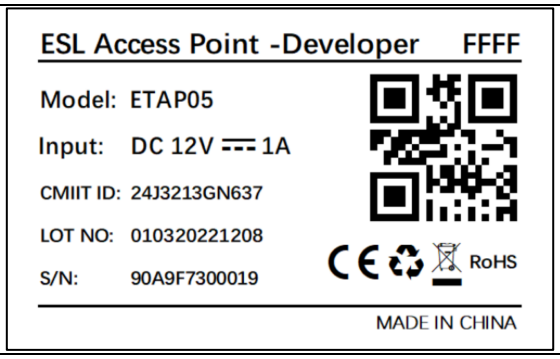
The eStation IoT device (type code AP05) is designed for system integrators to rapidly develop business projects and communicates using the MQTT v5 protocol.

AP04 vs. AP05:
Before using eStation, carefully review the differences between AP04 and AP05. Both use MQTT as the communication protocol, but their topics and data-structure definitions differ slightly.

AP05 Standard vs. Developer Editions:
Both the Developer Edition for system integrators and the Standard Edition for eRetail 3.2/4.0 use the same 4-digit ID scheme; however, their MQTT topics and data-structure definitions are different. This document covers only the Developer Edition.
You can distinguish the versions by the nameplate label on the back of the device.
The appearance differences among AP04 Developer Edition, AP05, and AP05 Developer Edition are as follows:

eStation Developer Edition （AP04）	
eStation (AP05)	

Limited Scope of Use

eStation Developer Edition （AP05）	 ESL Access Point -Developer FFFF Model: ETAP05 Input: DC 12V ⎓ 1A CMIIT ID: 24J3213GN637 LOT NO: 010320221208 S/N: 90A9F7300019 RoHS MADE IN CHINA

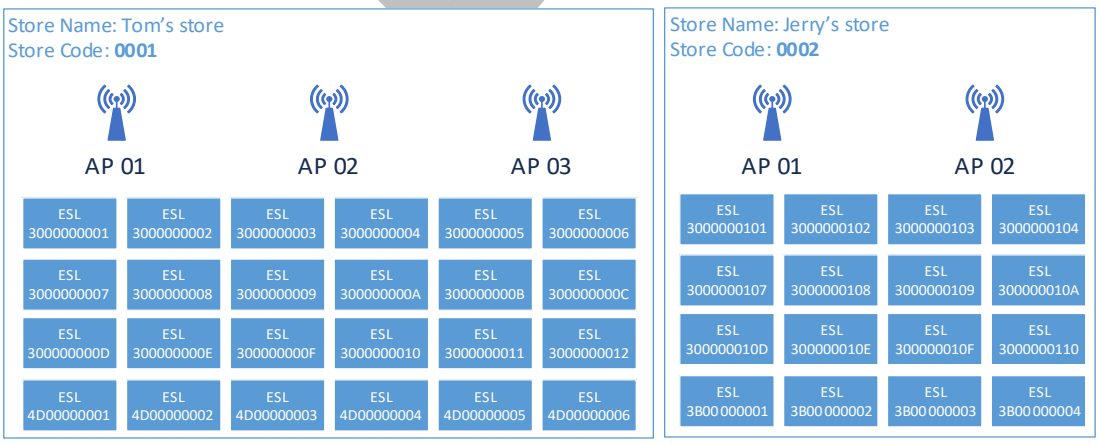
The eStation Developer Edition has a demo project available on GitHub:
https://github.com/andersonhwang/eStation_Developer_Edition

Note: The project is still under development. Currently, the .NET Console and WPF MVVM components are completed.

Before you start using eStation, you should understand the following key points:

1. AP (Access Point):
   The RF access point that connects to ESL devices.
2. AP ID:
   AP05 uses a 4-digit ID as its globally unique identifier. This ID is printed on the nameplate label located on the back of the device.
3. ESL (Electronic Shelf Label):
   Refers specifically to ESL Gen 3.0 devices that communicate via Bluetooth Low Energy (BLE 5.0).
4. DSL (Docked Shelf Label):
   Shelf labels powered via a docking rail. Currently limited to 2.4-inch TFT labels.
5. MQTT:
   eStation (AP05) uses the MQTT communication protocol.
6. Base64String:
   Images are converted into Base64String.
   For more details, refer to: Base64 Encode and Decode – Online.
7. MessagePack:
   eStation (AP05) uses MessagePack data format to reduce packet size.
   For more details, refer to: MessagePack: It's like JSON, but fast and small. (msgpack.org).

Example:
If your project includes two stores, it can be represented as follows:



Limited Scope of Use

The data structure should be:

Store Code	AP ID	ESL ID
0001	01	300000000001, 300000000002, 300000000007…
	02	300000000003, 300000000004, 300000000009 …
	03	300000000005, 300000000006, 30000000000B…
0002	01	300000000101, 300000000102, 300000000107…
	02	300000000103, 300000000104, 300000000109…

Note: Typically, your application should remember the AP that successfully communicated with each ESL. However, physically, an ESL is not bound to any specific AP. It only maintains, at a logical level, the ID of the last default AP that successfully sent data to it.

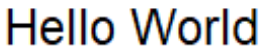
1.2 System Architecture of eStation

Basically, your project will send tasks containing the ESL ID and image data (Base64String) to eStation. eStation will then use 2.4 GHz RF to deliver the image data to the specified ESL, and return the result back to your project. In other words, eStation operates as the intermediary between the application and the ESL.

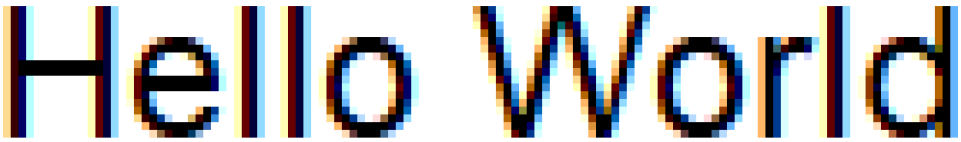
This document will cover two aspects: the application side and the eStation device side. Remember, if your application and the site are not on the same dedicated network, you should configure a strong password to secure the connection and add an X.509 certificate.

1.3 Images

Since electronic ESL screens are purely black-and-white or black-white-red (yellow) displays, they do not support grayscale.If your image looks like this:



When magnified by 800%, it appears as follows:



After removing the grayscale portions, it looks like this:

Limited Scope of Use

Hello World

The larger the font, the more visible the jagged edges become.
It is not recommended to use ESLs for displaying images. If image display is required, developers should apply dithering to simulate grayscale effects.

An example of a C# implementation for image dithering can be found here: Even more algorithms for dithering images using C# – Articles and information on C# and .NET development Topics • Cyotek.

For instance, an image may look like this:

The image output with and without dithering is shown as follows

Limited Scope of Use

2 Use eStation

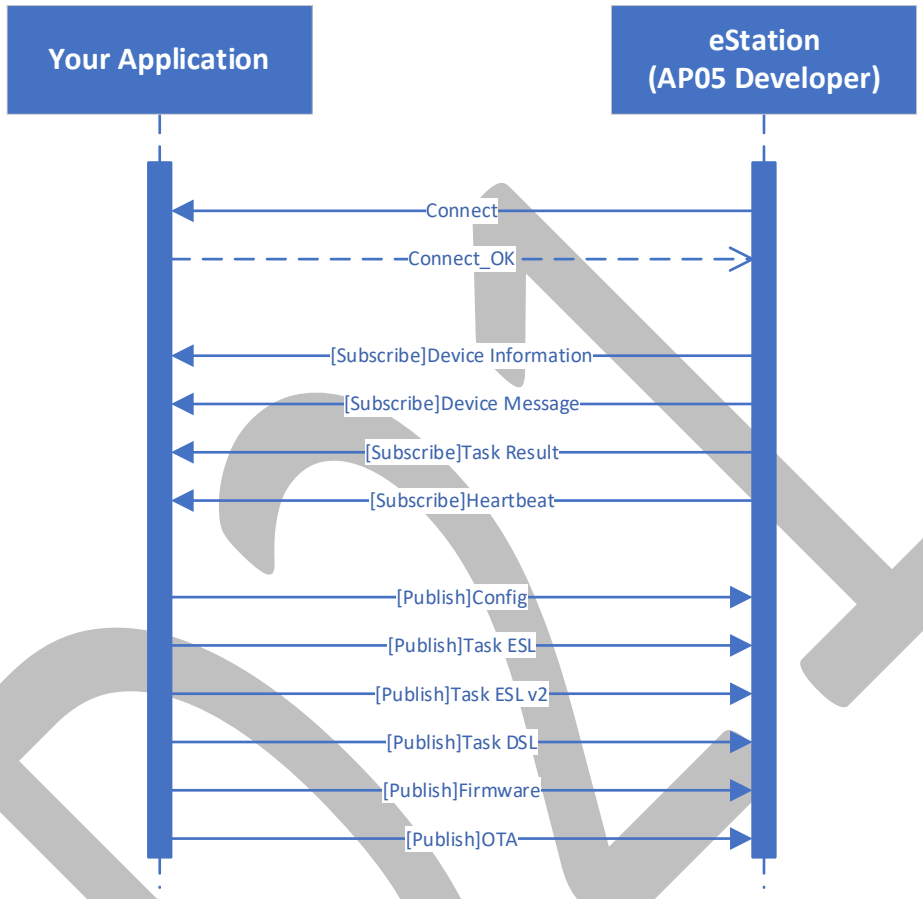
As noted earlier, there is no code-level integration with eStation. You must manage connections and communications within your application.
There are 10 topics in total: 4 topics published by the eStation device side and 6 topics published by the application side.
On the application side, the eStation device will subscribe to the following topics (Topic Aliases):

1. config (0x01): Use this topic to configure eStation.
2. taskESL (0x02): Publish ESL tasks with this topic (image data as Base64 string). — Deprecated, kept only for backward compatibility.
3. taskESL2 (0x03): Publish ESL tasks with this topic (image data as a byte array). — Recommended.
4. taskDSL (0x04): Publish DSL task configurations with this topic (currently limited to 2.4” TFT rail-powered labels).
5. firmware (0x05): Use this topic to OTA eStation or to pre-stage ESL/DSL firmware for later use by topic 0x06 (ota).
6. ota (0x06): Use this topic to OTA ESL and DSL.

Limited Scope of Use

From the eStation device side, your application should subscribe to the following topics (Topic Aliases):

1. infor (0x80): Reports device information after eStation establishes a connection to the service side.
2. message (0x81): Returns message codes when events occur.
3. result (0x82): Returns results after eStation communicates with an ESL.
4. heartbeat (0x83): Periodically reports current status.



Note: Publish/Subscribe is from your application side

2.1 Connect to eStation

eStation operates as an IoT device on the client side. After being powered on, it will attempt to connect to your application (server side).
The default parameters of eStation are:

Default target server address	192.168.1.92:9081
Default username	test
Default password	123456

Note: Please change the default username and password as soon as possible.

After connecting to a site, you can modify these settings using the configuration topic (see 2.7 Publish Configuration Information).
However, if you cannot connect to eStation, or if you forget its target server address, or the username/password, you can press and hold the reset button for a few seconds. Once you

Limited Scope of Use

see the REST indicator and the boot display, the device will revert to its default target server IP address, username, and password.

Note: If you reset eStation but it still cannot connect to your application, please check your firewall and network cable connection.

2.2 Receiving eStation Information

Interface Description：

Topic Alias	0x80
Topic	{Prefix}/{ID}/infor
Prefix	Topic prefix, default is /estation (without quotes).
ID	eStation ID
QoS Level	ExatclyOnce
PayloadSegment	The number of bytes of the eStationInfor object, serialized with MessagePack.

eStationInfor fields are:

#	Attribute	Type	Remark
0	ID	Char (4)	Unique code， 4-digit ID
1	Alias	String	NIckname， When set to two digits, the LED display will show it
2	IP	IP String	IP local address
3	MAC	MAC String	MAC address
4	ApType	Int	Fixed 4
5	ApVersion	String	Main Firmware Version
6	ModVersion	String	Module Firmware Version
7	DiskSize	Int	Disk Space (MB)
8	FreeSpace	Int	Available Space (MB)
9	Server	String	Target server address, can be either an IP address or a domain name.
10	ConnParam	String Array	Connection Parameters
11	Encrypt	Bool	True: Enable TLS 1.2 security protocol False: Disable TLS 1.2 security protocol (Default is disabled; see 2.9.2 X.509 Certificate)
12	AutoIP	Bool	Obtain IP Address Automatically (DHCP)
13	LocalIP	IP String	Static IP Address
14	Subnet	IP String	Subnet Mask
15	Gateway	IP String	Gateway
16	Heartbeat	Int	Heartbeat (seconds), minimum 15

2.3 Receiving eStation Messages

Interface Description：

Topic Alias	0x81
Topic	/estation/{ID}/message
ID	eStation ID
QoS Level	AtMostOnce

Limited Scope of Use

PayloadSegment	The number of bytes of the eStationInfor object, serialized with MessagePack.

eStationMessage Fields:

#	Attribute	Type	Remark
0	Code	MessageCode	Enumeration Type

The enumeration values of MessageCode are as follows:

- OK – General acknowledgment
- Idle – Idle state
- Result – Communication result returned
- Heartbeat – Heartbeat feedback
- ModError – Communication module error
- AppError – Main program error
- Busy – Device busy
- MaxLimit – Current data queue limit reached (pending tasks exceed 10,000 items or memory usage exceeds 512 MB)
- InvalidTaskESL – Invalid ESL task data
- InvalidTaskDSL – Invalid DSL task data
- InvalidConfig – Invalid configuration data
- InvalidOTA – Invalid OTA data

2.4 Receiving eStation Results

Interface Description：

Topic Alias	0x82
Topic	/estation/{ID}/result
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	The number of bytes of the TaskResult object, serialized with MessagePack. The TaskResult object contains a list of TagResult objects.

TaskResult Field:

#	Attribute	Type	Remark
0	Port	Int	MOD Serial Port Number
1	WaitCount	Int	Total Task Count in Cache
2	SendCount	Int	Sent Task Count (in RF module)
3	Message	String	Reserved Field
4	Tags	List<TagResult>	ESL Result List

TagResult Fields:

#	Attribute	Type	Remark
0	TagID	String	ESL ID
1	RfPower	Int	RF Power， dBm, -256 Indicates No data
2	Battery	Byte	Battery Voltage(V)， V， 0 indiates no data
3	Version	Byte	Electronic ESL firmware version， reserved for future use
4	Status	Byte	Status， reserved for future use
5	Token	Int	Token， used for data loopback， see 2.5 token
6	Temperature	Int	Temperature， ℃， 0 indicates no data
7	Channel	Int	channel， reserved for future use

Limited Scope of Use

8	UtcTime	Byte Array	UTC time, reserved for future use, DSL only
9	TimePercent	Byte Array	Time reliability, reserved for future use, DSL only
10	Count	Byte	Counter, reserved for future use, DSL only

Note: There is an internal queue inside the workstation. When the application sends task data to the workstation, if the RF module is idle, the data will be sent immediately; otherwise, if the RF module is busy, the data will wait until the RF module becomes idle. Therefore, communication between the application and eStation is asynchronous.

2.5 Receiving eStation Heartbeat

Interface Description:

Topic Alias	0x83
Topic	/estation/{ID}/heartbeat
ID	eStation ID
QoS Level	AtMostOnce
PayloadSegment	The number of bytes of the ApHeartbeat object, serialized with MessagePack. The ApHeartbeat object contains a list of TagHeartbeat objects.

ApHeartbeat Fields:

#	Attribute	Type	Remark
0	Id	String	AP ID
1	ConfigVersion	Int	Configuration Version (default 0, reserved for future use)
2	ApVersion	String	Base Station Version
3	ModVersion	String	Bluetooth Module Version
4	Message	MessageCode	Message, see 2.3 MessageCode
5	MessageEx	String	Message (Extended Field)
6	WaitCount	Int	Current Number of Queued Tags
7	SendCount	Int	Current Number of Tags in Communication
8	Tags	TagHeartbeat list	Tag Heartbeat Information

TagHeartbeat Fields:

#	Attribute	Type	Remark
0	TagId	String	Tag ESL/DSL ID
1	RfPower	Int	RF Power, dBm, -256 indicates no data
2	Battery	Byte	Battery Voltage, V, 0 indicates no data
3	Version	Byte	Electronic ESL Firmware Version, reserved for future use
4	Status	Byte	Status, reserved for future use
5	Token	Int	Token, used for data loopback, see 2.5 Token
6	Temperature	Int	Temperature, °C, 0 indicates no data
7	Channel	Int	Channel, reserved for future use
8	UtcTime	Byte Array	UTC Time, DSL only
9	TimePercent	Byte	Time Reliability, DSL only
10	Count	Byte	Counter, DSL only
11	Factory	Byte	Factory Code, used for OTA
12	Color	Byte	Screen Color Code, used for OTA
13	Size	Byte	Screen Size Code, used for OTA

Limited Scope of Use

14	Type	Byte	Screen Type Code, used for OTA

Note: The data structures of heartbeat and result are similar.
The difference is that heartbeat contains information detected from the label's heartbeat, whereas result represents the outcome returned after your application has sent data. The former occurs proactively, while the latter occurs reactively.

2.6 Publishing Configuration Information

Interface Description：

Topic Alias	0x01
Topic	/estation/{ID}/configure
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	The number of bytes of the eStationConfig object, serialized with MessagePack.

eStationConfig Fields:

#	Attribute	Type	Remark
0	Alias	String	Nickname – when set to two digits, it will be displayed on the LED display
1	Server	String	Target server address
2	ConnParam	String Array	Connection parameters
3	Encrypt	Bool	TLS 1.2 security protocol: True = enabled, False = disabled (default is disabled; see 2.9.2 X.509 Certificate)
4	AutoIP	Bool	Obtain IP address automatically (DHCP)
5	LocalIP	IP String	Static IP address
6	Subnet	IP String	Subnet mask
7	Gateway	IP String	Gateway
8	Heartbeat	Int	Heartbeat (seconds), minimum 15

2.7 Publishing ESL Task （Base64 Version）

Note: This topic is deprecated. It is recommended to use the Bytes version (see 2.8) instead.

Interface Description：

Topic Alias	0x02
Topic	/estation/{ID}/taskESL
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list array of ESLEntity objects, serialized with MessagePack.

ESLEntity Attribute:

#	Attribute	Type	Remark
0	TagID	String	ESL ID
1	Pattern	Int, Pattern	Pattern Code, see 3.1 Pattern
2	PageIndex	Int, PageIndex	Page Index, see 3.2 PageIndex
3	R	Bool	Red LED
4	G	Bool	Green LED
5	B	Bool	Blue LED

Limited Scope of Use

6	Times	Int, 2-byte range	LED Blink Count, in seconds
7	Token	Int, 2-byte range	Token, will be returned in 2.3 Token
8	CurrentKey	Int, 2-byte range	Current key, leave empty if using the default key
9	NewKey	8 Byte	New key, leave empty if not required
10	Base64String	String	Base64String value of the image in Bitmap format

Override: For a given ESL ID, if your application does not wait for the result of the previous task and continues sending new task data, eStation will discard the previous task.
Pixel Format: The pixel format must be BGRA; otherwise, color parsing errors or abnormal display may occur.
Dimensions: eStation checks the image data size (Base64String) in the ESL task payload. If the size is incorrect (or if the actual image dimensions do not match the label screen size or padded dimensions), the update will fail. It is not possible to infer the actual dimensions from the byte array alone—for example, a 300×400 image and a 200×600 image produce the same bitmap data length.
For details on label types and screen resolutions, see 3.1 ESL Gen 3.0 Model List.
Alignment: Because the screen data width must be a multiple of 8, and some models may not meet this requirement, you can either send data with the exact screen dimensions or pad the width up to the nearest multiple of 8 (the extra part will not be processed or displayed).
Example: For the ET0213 label, both 250×122 and 256×122 images are acceptable, but only the 250×122 portion will be displayed.
Limitations: When pushing data via Base64 strings, the eStation task buffer limit is low (for example, with a 4.2” tri-color label, the limit may be ~800 tasks). To increase this limit, use the Bytes version (see 2.7).
Keys: For details on current and new keys, see 2.12 Secure Communication.

2.8 Publishing ESL Task （Bytes Version）

Interface Description：

Topic Alias	0x03
Topic	/estation/{ID}/taskESL2
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list array of ESLEntity2 objects, serialized with MessagePack.

ESLEntity2 Fields:

#	Attribute	Type	Remark
0	TagID	String	ESL ID
1	Pattern	Int, Pattern	Pattern Code, see 3.1 Pattern
2	PageIndex	Int, PageIndex	Page Index, see 3.2 PageIndex
3	R	Bool	Red LED
4	G	Bool	Green LED
5	B	Bool	Blue LED
6	Times	2 Byte	LED Blink Count, seconds
7	Token	2 Byte	Token, will be returned in 2.3 Token
8	CurrentKey	8 Byte	Current key, leave empty if using the default key
9	NewKey	8 Byte	New key, leave empty if not required

Limited Scope of Use

10	Bytes	Byte Array	Byte array of the image in Bitmap format
11	Compress	Bool	Compression enabled flag, default is True, using GZip compression algorithm

Pixel Format: The pixel requirements for this topic are the same as those in 2.7 Publishing ESL Task (Base64 Version).
Dimensions: The image dimension requirements for this topic are also the same as in 2.7 Publishing ESL Task (Base64 Version).
Limit: When using the Bytes version topic, the current upper limit for eStation is 20,000 tasks (sufficient for most scenarios). Any tasks beyond this limit will be discarded, and a MessageCode.MaxLimit will be returned via the topic described in 2.3 Receiving eStation Messages.
Image Data: Supports pixel byte arrays in Bitmap format (both RGBA32 and RGB24 modes), and also supports file data from mainstream image formats. GZip compression is supported; in this case, the Compress parameter must be set to True. For image size handling, unlike taskESL, the image dimensions must strictly match the screen pixel specifications defined by the product model.

2.9 Publishing DSL Task

Interface Description：

Topic Alias	0x04
Topic	/estation/{ID}/taskDSL
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list of array of DSLEntity objects， serialized with MessagePack

DSLEntity Fields:

#	Attribute	Type	Remark
0	TagID	String	ESL ID
1	R	Bool	Red LED
2	G	Bool	Green LED
3	B	Bool	Blue LED
4	Period	2 Byte	LED Blink Count, -1 to 3600, default 3600, unit: times
5	Interval	2 Byte	Light Interval, 100–10,000, default 1000, unit: ms
6	Duration	8 Byte	Light Duration, 50–100, default 50, unit: ms
7	Token	8 Byte	Token, will be returned in 2.3 Token
8	HexData	Byte Array	Bin file data stream

Data: A .bin file refers to an image/instruction file specifically generated for 2.4” TFT shelf labels (DSL).

Constant Light: When Period = -1, the label remains constantly lit (in this mode, Interval and Duration are ignored).

2.10 Publishing DSL Task （with Pattern and Key）

Interface Description：

Limited Scope of Use

Topic Alias	0x07
Topic	/estation/{ID}/taskDSL2
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list array of DSLEntity2 objects, serialized with MessagePack.

DSLEntity2 Fields:

#	Attribute	Type	Remark
0	TagID	String	ESL ID
1	R	Bool	Red LED
2	G	Bool	Green LED
3	B	Bool	Blue LED
4	Period	2 Byte	LED Blink Count, -1~3600, Default 3600， Unit: times
5	Interval	2 Byte	Light Interval, 100~10000, default 1000， unit s
6	Duration	8 Byte	Light Duration, 50~100, default 50， unit ms
7	Token	8 Byte	Token， will be returned in 2.3 Token
8	HexData	Byte Array	Bin file data stream
9	Pattern	Int, Pattern	Pattern code, see 3.1 Pattern
10	CurrentKey	8 Byte	Current Key， Leave empty if using the default key
11	NewKey	8 Byte	New Key， Leave empty if not required

Data: A .bin file refers to an image/instruction file specifically generated for 2.4” TFT shelf labels (DSL).

Constant Light: When Period = -1, the label remains constantly lit (in this mode, Interval and Duration are ignored).

2.11 Publishing OTA Task （Firmware）

Interface Description：

Topic Alias	0x05
Topic	/estation/{ID}/firmware
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list array of OTAData objects, serialized with MessagePack

OTAData Fields:

#	Attribute	Type	Remark
0	DownloadUrl	String	Firmware download URL
1	ConfirmUrl	String	Target Server Address
2	Type	Int	OTA Type： 0-eStation, 1/2-MOD, 3-ESL/DSL
3	Version	String	Version number
4	Name	String	Firmware name
5	MD5	IP String	Static IP address

Note:
If Type = 0, the topic is treated as eStation firmware. After the download and verification are complete, eStation will immediately reboot and perform the OTA process.
If Type = 1 or 2, it is treated as MOD firmware, and a MOD OTA process will start immediately.

Limited Scope of Use

If Type = 3, it is considered pre-staged ESL/DSL firmware, which will later be used for label OTA as described in section 2.11.
Once OTA begins, do not communicate with eStation until the OTA process has fully completed.
MD5 format:
In earlier versions (< 1.0.28): 00-11-22-33-44-55-66-77
In later versions (≥ 1.0.28): 00111223344556677

2.12 Publishing OTA Task（ESL）

Interface Description：

Topic Alias	0x06
Topic	/estation/{ID}/ota
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	A list array of OTATask objects, serialized with MessagePack.

OTATask Fields:

#	Attribute	Type	Remark
0	Step	Int	Type：0-PreOTA; 1-OTA; 2-Clean
1	Firmware	String	Firmware file name
2	TagIDList	String Array	List of tag IDs to be updated

Note:
This topic depends on the firmware described in Section 2.10; otherwise, the firmware cannot be located and the update cannot proceed.
This topic must be executed at least three consecutive times (with Step = 0 / 1 / 2). After each step, check the results as described in Section 2.4 to ensure that all tags in the current step have communicated successfully before proceeding to the next step.
The TagIDList in this topic must consist of IDs belonging to the same model of tags, and Firmware must be the 2-digit code corresponding to that model.

2.13 Secure Communication

2.12.1. Tag Key

In general, if other projects use their eStation (AP) to control your project's ESLs, unexpected and unacceptable results may occur. In such cases, it is recommended to set a key for your ESLs.

When you receive new ESLs from the sales manager, the default key is empty (11 bytes, i.e., FFFFFFFFFFFFFFFF). You may either keep this default or configure both a current key and a new key. Afterward, all communication should use the new key.

1. The key has a fixed length of 22 characters (11 bytes).
2. Only hexadecimal characters (0–9, A–F) are accepted.
3. If the current key is empty, eStation will use the default key as a replacement.
4. If the New Key is empty, the workstation will not change the key. If the New Key is provided, the base station will update the key.

Limited Scope of Use

5. Important: Store your keys securely. If you forget a key, you will permanently lose control of the ESL.
6. Only ESLs with firmware version 35 or later support the key communication feature.

2.12.2. X.509 Certificate

If TLS 1.2 security protocol is enabled in 2.7 Publishing Configuration Information, the server can use a custom X.509 certificate to encrypt communication data. How the server uses the X.509 certificate and TLS 1.2 protocol depends on the server-side development language and framework chosen by the developer.

For example, if the server is developed using C# with the MQTTnet framework, a .pfx certificate is required.
Sample code is as follows：

```
public async Task Run()
{
   var options = new MqttServerOptionsBuilder()
      .WithDefaultEndpoint()
      .WithEncryptedEndpoint()
      .WithEncryptedEndpointPort(PORT) // TCP port in your server side
      .WithEncryptionCertificate(CERTIFICATE_PATH) // Your certificate path
      .WithEncryptionSslProtocol(SslProtocols.Tls12)
      .Build();
   _server = _factory.CreateMqttServer(options);
   _server.ValidatingConnectionAsync += ServerOnValidatingConnectionAsync;
        _server.ClientConnectedAsync
           += ServeOnClientConnectedAsync;
        _server.ClientDisconnectedAsync
           += ServerOnClientDisconnectedAsync;
        _server.InterceptingPublishAsync +=
           ServerOnInterceptingPublishAsync;
   await _server.StartAsync();
}
```

3 References

3.1 ESL Gen 3.0 Model List

Type	Size (inch)	Screen	Color	Resolution (H*W)	Freezer-Compatible
ET0154-33	1.54	Eink	B/W/R	200*200	-
ET0213-36	2.13	Eink	B/W/R	250*122	-
ET0213-39	2.13	Eink	B/W	250*122	Y
ET0266-3A	2.66	Eink	B/W/R	296*152	-
ET0266-5B	2.66	Eink	B/W	296*152	Y
ET0290-3D	2.9	Eink	B/W/R	296*128	-
ET0290-3F	2.9	Eink	B/W	296*128	-
ET0290-54	2.9	Eink	B/W	296*128	Y
ET0420-40	4.2	Eink	B/W/R	400*300	-

Limited Scope of Use

ET0420-42	4.2	Eink	B/W	400*300	-
ET0750-44	7.5	Eink	B/W/R	800*480	-
ET0750-46	7.5	Eink	B/W	800*480	-
ET0430-4C	4.3	Eink	B/W/R	522*152	-
ET0580-4F	5.8	Eink	B/W/R	648*480	-
ET0350-55	3.5	Eink	B/W/R	384*184	-
ET1250-58	12.5	Eink	B/W/R	1304*984	-
ET0420-5D	4.2	TFT	B/W/R	400*300	-
ET1020-64	10.2	Eink	B/W/R	960*640	-
ET1020-67	10.20	Eink	B/W	960*640	-
ET1330-68	13.3	Eink	B/W/R	960*680	-
ET0579-6F	5.8	Eink	B/W/R	792*272	-
ET0154H-71	1.54	Eink	B/W/R	200*200	-
ET0130H-73	1.3	Eink	B/W/R	200*144	-
ET0154-80	1.54	Eink	B/W/R/Y	200*200	-
ET0213-81	2.13	Eink	B/W/R/Y	250*122	-
ET0266-82	2.66	Eink	B/W/R/Y	296*152	-
ET0266-83	2.66	Eink	B/W/R/Y	360*184	-
ET0290-84	2.9	Eink	B/W/R/Y	296*128	-
ET0290-85	2.9	Eink	B/W/R/Y	384*168	-
ET0350-86	3.5	Eink	B/W/R/Y	384*184	-
ET0420-87	4.2	Eink	B/W/R/Y	400*300	-
ET0580-88	5.8	Eink	B/W/R/Y	648*480	-
ET0750-89	7.5	Eink	B/W/R/Y	800*480	-
ET0750-8A	7.5	Eink	B/W/R/Y	880*528	-
ET1020-8B	10.2	Eink	B/W/R/Y	960*640	-
ET0097H-8C	0.97	Eink	B/W/R/Y	184*88	-
ET0130H-8D	1.3	Eink	B/W/R/Y	200*144	-
ET1330-8E	13.3	Eink	B/W/R/Y	960*680	-
ET0370-8F	3.7	Eink	B/W/R/Y	416*240	-
ET0154H-90	1.54	Eink	B/W/R/Y	200*200	-
ET0430-91	4.3	Eink	B/W/R/Y	522*152	-
ET0730-A0	7.3	Eink	6-Color	800*480	-
ET1330-A1	13.3	Eink	6-Color	1600*1200	-
ET0240C-D0	2.4	TFT	Full Color?	320*240	-

Note: In the color column, B stands for Black, W stands for White, R stands for Red, and Y stands for Yellow.

Limited Scope of Use

3.2 Pattern

Pattern	Description
0- UpdateDisplay	Update and Display
1- Update	Update and not display
2- Display	Display
3- LED	Blink LED
4- Key	Update Key（see 2.12.1 Tag Key）
5- ClearGif	Clear Animation（DSL type only）
6- ClearGifUpdateImage	Clear Animation and Update Static Image，（DSL type only）

3.3 PageIndex

Page ID	Description
0- P0	Page 1
1- P1	Page 2
2- P2	Page 3
3- P3	Page 4
4- P4	Page 5，The compressed image size must not exceed 32 KB.
5- P5	Page 6，The compressed image size must not exceed 32 KB.
6- P6	Page 7，The compressed image size must not exceed 32 KB.
7- P7	Page 8，The compressed image size must not exceed 32 KB.

Note: Currently, all ESLs have only 8 pages of data cache. Due to storage limitations, images delivered to pages 5–8 must not exceed 32 KB after compression (using the LZSS compression algorithm, implemented internally by the base station; the server still transmits the original image).

3.4 RGB Color Definitions

The ESL LED uses an RGB tri-color light, which can visually produce seven color effects.

Color	R	G	B
Off	×	×	×
Blue	×	×	√
Green	×	√	×
Cyan	×	√	√
Red	√	×	×
Purple	√	×	√
Yellow	√	√	×
White	√	√	√

3.5 Data Definitions（C#）

eStationInfor：Used to define the configuration, status, and parameters of eStation. Among them, ID, MAC, AppVersion, DummyVersion, TotalCount, and SendCount are read-only. Its definition is as follows:

Limited Scope of Use

```
/// <summary>
/// AP information
/// </summary>
[MessagePackObject]
public class eStationInfor
{
    /// <summary>
    /// ID: Code4
    /// </summary>
    [Key(0)]
    public string ID { get; set; } = "0000";
    /// <summary>
    /// Alias
    /// </summary>
    [Key(1)]
    public string Alias { get; set; } = "";
    /// <summary>
    /// Local IP
    /// </summary>
    [Key(2)]
    public string IP { get; set; } = "127.0.0.1";
    /// <summary>
    /// MAC, refer ID
    /// </summary>
    [Key(3)]
    public string MAC { get; set; } = "90A9F7300000";
    /// <summary>
    /// AP type
    /// </summary>
    [Key(4)]
    public int ApType { get; set; } = 4; // AP05 = 3;
    /// <summary>
    /// Firmware version
    /// </summary>
    [Key(5)]
    public string ApVersion { get; set; } = "1.0.1";
    /// <summary>
    /// MOD ModVersion
    /// </summary>
    [Key(6)]
    public string ModVersion { get; set; } = "";
    /// <summary>
    /// Disk size (MB)
    /// </summary>
    [Key(7)]
    public int DiskSize { get; set; } = 0;
    /// <summary>
    /// Disk free space (MB)
    /// </summary>
    [Key(8)]
    public int FreeSpace { get; set; } = 0;
    /// <summary>
    /// Server address
    /// </summary>
    [Key(9)]
    public string Server { get; set; } = "192.168.4.92";
    /// <summary>
    /// Connection parameters
```


Limited Scope of Use

```
    /// </summary>
    [Key(10)]
    public string[] ConnParam { get; set; } = Array.Empty<string>();
    /// <summary>
    /// Auto network
    /// </summary>
    [Key(11)]
    public bool AutoIP { get; set; } = true;
    /// <summary>
    /// Local IP, empty if auto network
    /// </summary>
    [Key(12)]
    public string LocalIP { get; set; } = string.Empty;
    /// <summary>
    /// Subnet mask, empty if auto network
    /// </summary>
    [Key(13)]
    public string Subnet { get; set; } = string.Empty;
    /// <summary>
    /// Gateway, empty if auto network
    /// </summary>
    [Key(14)]
    public string Gateway { get; set; } = string.Empty;
    /// <summary>
    /// Heartbeat speed
    /// </summary>
    [Key(15)]
    public int Heartbeat { get; set; } = 15;
}
```

eStationConfig： Used to define the configuration information of the base station.The definition is as follows：

```
[MessagePackObject]
internal class eStationConfig
{
    /// <summary>
    /// Alias
    /// </summary>
    [Key(0)]
    public string Alias { get; set; } = "";
    /// <summary>
    /// Server address
    /// </summary>
    [Key(1)]
    public string Server { get; set; } = "192.168.4.92";
    /// <summary>
    /// Connection parameters
    /// </summary>
    [Key(2)]
    public string[] ConnParam { get; set; } = Array.Empty<string>();
    /// <summary>
    /// Auto network
    /// </summary>
    [Key(3)]
    public bool AutoIP { get; set; } = true;
    /// <summary>
    /// Local IP, empty if auto network
    /// </summary>
    [Key(4)]
    public string LocalIP { get; set; } = string.Empty;
```


Limited Scope of Use

```
    /// <summary>
    /// Subnet mask, empty if auto network
    /// </summary>
    [Key(5)]
    public string Subnet { get; set; } = string.Empty;
    /// <summary>
    /// Gateway, empty if auto network
    /// </summary>
    [Key(6)]
    public string Gateway { get; set; } = string.Empty;
    /// <summary>
    /// Heartbeat speed
    /// </summary>
    [Key(7)]
    public int Heartbeat { get; set; } = 15;
}
```

eStationMessage： Used to define the message of the base station.The definition is as follows：

```
[MessagePackObject]
public class eStationMessage
{
    /// <summary>
    /// Message code
    /// </summary>
    [Key(0)]
    public MessageCode Code { get; set; } = MessageCode.OK;
}
```

ESLEntity： Used to define the task data information of the ESL. The definition is as follows：

```
/// <summary>
/// ESL entity
/// </summary>
[MessagePackObject]
public class ESLEntity
{
    [Key(0)]
    public string TagID { get; set; }
    [Key(1)]
    public Pattern Pattern { get; set; }
    [Key(2)]
    public PageIndex PageIndex { get; set; }
    [Key(3)]
    public bool R { get; set; }
    [Key(4)]
    public bool G { get; set; }
    [Key(5)]
    public bool B { get; set; }
    [Key(6)]
    public int Times { get; set; }
    [Key(7)]
    public int Token { get; set; }
    [Key(8)]
    public string OldKey { get; set; }
    [Key(9)]
    public string NewKey { get; set; }
    [Key(10)]
    public string Base64String { get; set; } = "";
}
```


Limited Scope of Use

TaskResult： Used to define the data information of a set of ESL/PLT communication. The definition is as follows：

```
/// <summary>
/// Task result entity
/// </summary>
[MessagePackObject]
public class TaskResult
{
    /// <summary>
    /// COM Port
    /// </summary>
    [Key(0)]
    public int Port { get; set; } = 0;

    /// <summary>
    /// Total count in cache
    /// </summary>
    [Key(1)]
    public int TotalCount { get; set; }

    /// <summary>
    /// Current sending count
    /// </summary>
    [Key(2)]
    public int SendCount { get; set; }

    /// <summary>
    /// Message
    /// </summary>
    [Key(3)]
    public MessageCode Message { get; set; } = 0;

    /// <summary>
    /// Tag results list
    /// </summary>
    [Key(4)]
    public List<TagResult> Tags { get; set; } = new List<TagResult>();
}
```

TagResult： Used to define the data information of a s singel ESL/PTL communication result. The definition is as follows:

```
[MessagePackObject]
/// <summary>
/// Tag result
/// </summary>
public class TagResult
{
    /// <summary>
    /// Tag ID
    /// </summary>
    [Key(0)]
    public string TagID { get; set; } = string.Empty;
    /// <summary>
    /// RF power
    /// </summary>
    [Key(1)]
    public int RfPower { get; set; } = -256;
    /// <summary>
    /// Battery
```


Limited Scope of Use

```
    /// </summary>
    [Key(2)]
    public byte Battery { get; set; } = 0;
    /// <summary>
    /// Version
    /// </summary>
    [Key(3)]
    public byte Version { get; set; } = 0;
    /// <summary>
    /// Status
    /// </summary>
    [Key(4)]
    public byte Status { get; set; } = 0;
    /// <summary>
    /// Token
    /// </summary>
    [Key(5)]
    public int Token { get; set; } = 0;
    /// <summary>
    /// Temperature
    /// </summary>
    [Key(6)]
    public int Temperature { get; set; } = 0;
    /// <summary>
    /// Channel
    /// </summary>
    [Key(7)]
    public int Channel { get; set; } = 0;
    /// <summary>
    /// [DSL]UTC time,
    /// </summary>
    [Key(8)]
    public byte[] UtcTime { get; set; } = Array.Empty<byte>();
    /// <summary>
    /// [DSL]UTC percent,
    /// </summary>
    [Key(9)]
    public byte[] TimePercent { get; set; } = Array.Empty<byte>();
    /// <summary>
    /// [DSL]Count
    /// </summary>
    [Key(10)]
    public byte Count { get; set; } = 0;
}
```

■

Limited Scope of Use